Saarthi Manual

Section 5: Real-Time Object Detection + Obstacle Alerts (AI + Sensors + Audio)

🎯 5.1 OBJECTIVE

By the end of this section, you will have:
✅ A working Python script that

- Uses **TFLite model + Coral USB** to detect objects
- Reads ultrasonic sensors to avoid close-range obstacles
- Plays **spoken alerts** using `espeak` or `pyttsx3`
  ✅ A **modular codebase** that is easy to modify and expand

📁 5.2 FOLDER STRUCTURE FOR SAARTHI CODE

Create a project folder like this:

```
/home/pi/saarthi/
│
├── models/
│   └── mobilenet_ssd_v2.tflite
│
├── labels.txt
├── main.py
├── camera.py
├── sensor.py
├── audio.py
└── utils.py
```

🎁 5.3 DOWNLOAD TFLITE MODEL AND LABELS

1. Create `models/` directory:

```
mkdir -p /home/pi/saarthi/models
cd /home/pi/saarthi/models
```

2. Download MobileNet SSD v2 `.tflite` model optimized for Coral:

```
wget https://github.com/google-coral/edgetpu/raw/master/test_data/mobilenet_ssd_v2_coco_quant_postprocess_edgetpu.tflite
```

3. Download COCO Labels:

```
wget https://raw.githubusercontent.com/google-
coral/edgetpu/master/test_data/coco_labels.txt -O /home/pi/saarthi/labels.txt
```

□ 5.4 CREATE `sensor.py` – Ultrasonic Module

```
# sensor.py
import RPi.GPIO as GPIO
import time

class UltrasonicSensor:
    def __init__(self, trig, echo):
        self.trig = trig
        self.echo = echo
        GPIO.setmode(GPIO.BCM)
        GPIO.setup(self.trig, GPIO.OUT)
        GPIO.setup(self.echo, GPIO.IN)

    def get_distance(self):
        GPIO.output(self.trig, False)
        time.sleep(0.01)
        GPIO.output(self.trig, True)
        time.sleep(0.00001)
        GPIO.output(self.trig, False)

        while GPIO.input(self.echo) == 0:
            pulse_start = time.time()
        while GPIO.input(self.echo) == 1:
            pulse_end = time.time()

        pulse_duration = pulse_end - pulse_start
        distance = round(pulse_duration * 17150, 2)
        return distance
```

🔊 5.5 CREATE `audio.py` – Voice Alert Module

```
# audio.py
import os

def speak(text):
    os.system(f'espeak "{text}"')
```

✅ You can later switch to `pyttsx3` or `gTTS` if offline voices are not loud enough.

📷 5.6 CREATE `camera.py` – AI Object Detection Module

```
# camera.py
import cv2
import numpy as np
from tflite_runtime.interpreter import Interpreter, load_delegate

class ObjectDetector:
    def __init__(self, model_path, label_path):
        self.labels = self.load_labels(label_path)
```

```
        self.interpreter = Interpreter(model_path=model_path,
              experimental_delegates=[load_delegate('libedgetpu.so.1')])
        self.interpreter.allocate_tensors()
        self.input_details = self.interpreter.get_input_details()
        self.output_details = self.interpreter.get_output_details()

    def load_labels(self, path):
        with open(path, 'r') as f:
            return {i: line.strip() for i, line in enumerate(f.readlines())}

    def detect(self, frame):
        input_shape = self.input_details[0]['shape']
        image_resized = cv2.resize(frame, (input_shape[2], input_shape[1]))
        input_data = np.expand_dims(image_resized, axis=0)
        self.interpreter.set_tensor(self.input_details[0]['index'], input_data)
        self.interpreter.invoke()

        boxes = self.interpreter.get_tensor(self.output_details[0]['index'])[0]
        classes = self.interpreter.get_tensor(self.output_details[1]['index'])[0]
        scores = self.interpreter.get_tensor(self.output_details[2]['index'])[0]

        results = []
        for i in range(len(scores)):
            if scores[i] > 0.6:
                label = self.labels.get(int(classes[i]), 'Unknown')
                results.append(label)
        return results
```

□ 5.7 CREATE `main.py` – Full Integrated Logic

```
# main.py
import cv2
import time
from camera import ObjectDetector
from audio import speak
from sensor import UltrasonicSensor

# Paths
MODEL = "models/mobilenet_ssd_v2.tflite"
LABELS = "labels.txt"

# Init modules
detector = ObjectDetector(MODEL, LABELS)
front_sensor = UltrasonicSensor(trig=23, echo=24)
down_sensor = UltrasonicSensor(trig=17, echo=27)

# Open camera
cap = cv2.VideoCapture(0)

print("System running...")

last_spoken = None
while True:
    ret, frame = cap.read()
    if not ret:
        continue

    detected_objects = detector.detect(frame)
    front_distance = front_sensor.get_distance()
    down_distance = down_sensor.get_distance()
```

```
    # Speak object only once every 3 seconds
    if detected_objects:
        for obj in detected_objects:
            if obj != last_spoken:
                speak(f"{obj} ahead")
                last_spoken = obj
                time.sleep(3)

    # Handle downward danger
    if down_distance > 100:  # Empty space detected (like stairs down)
        speak("Stair down detected")
        time.sleep(3)

    # Close obstacle warning
    if front_distance < 50:
        speak("Obstacle very close")
        time.sleep(2)

    # Optional: display frame
    # cv2.imshow("View", frame)
    # if cv2.waitKey(1) & 0xFF == ord('q'):
    #     break
```

□ 5.8 TEST EVERYTHING TOGETHER

1. Navigate to your project folder:

```
cd /home/pi/saarthi
python3 main.py
```

2. Stand in front of the camera with a **bike, dog toy, or hand** — it should say:

“Person ahead” or “Dog ahead”

3. Place object < 40cm in front → it should say:

“Obstacle very close”

4. Raise the stick with downward sensor over stairs → it should say:

“Stair down detected”

✅ 5.9 MILESTONE COMPLETE

You now have a **fully working AI-powered obstacle detection + alert system** that:

- Recognizes real-world objects in front
- Warns about close range threats
- Detects downward stairs
- Speaks clear and accurate voice alerts

🔜 Next Steps (Section 6 Preview)

In the next section, we'll:

- Add better voice system (offline Indian-accent voices)
- Optimize object detection speed
- Improve alert frequency control (avoid repeating too much)
- Add optional GPS base (if you choose later)